개인 회고

진행하면서 막혔던 부분과 해결

실행하다가 실제로 막힌 것

막힌 부분	원인	해결
import 셀에서 `CLIPFeatureExtractor` 에러	코랩 기본 `transformers`(5.18)에서 이 이름이 빠졌다. `transformers` 만 낮추면 이번엔 `diffusers 0.40`과 `huggingface-hub` 버전이 충돌했다	코랩 터미널에서 조합을 시험해 `transformers<5` + `diffusers==0.30.3`으로 맞췄다
`%load_ext autoreload` 에러	코랩의 Python 3.13에는 이 확장이 쓰는 `imp` 모듈이 없다	코드를 고칠 때 다시 불러오는 개발용 기능이라 결과와 무관해서 주석 처리했다
정상 이미지를 불러오는 셀에 그림이 안 나옴	코랩에서는 PIL의 `show()`가 셀 아래에 그림을 띄우지 않는다	`display()`로 바꿨다

실행 전에 코드를 읽고 미리 고친 것

내용	근거	해결
설치 셀이 없음	저장소 코드가 `pyrallis`와 OpenAI `clip`을 쓰는데, 실행해 보니 둘 다 코랩에 없어서 새로 설치됐다	첫 셀에 설치 줄을 넣었다
PCB 셀이 읽는 `normal.png`가 저장소에 없음	저장소 파일 목록에 없다. 강사가 따로 넣어 둔 파일로 보인다	원본 노트북 출력에 남은 PCB 사진을 꺼내 `normal.png`로 저장하고 코랩에 올렸다

처음에는 모델 주소 `runwayml/stable-diffusion-v1-5`도 저장소가 사라진 줄 알고 실행해 보지 않은 채 미러 주소로 바꿨다. 나중에 확인해 보니 옛 주소가 새 주소로 자동 연결돼서 원본 줄 그대로 돌아갔고, 원본으로 되돌렸다. 고치기 전에 원본이 정말 안 되는지 먼저 돌려 봐야 한다는 것을 배웠다.

원본 코드는 그대로 두고, 바꾼 줄은 바로 위에 원본 줄과 바꾼 이유를 주석으로 남겼다.

실습을 통해 알게 된 것

공장에서는 불량이 드물어서 결함 사진을 모으기 어렵다. 이번 실습의 AnomalyAny는 정상 사진 1장과 결함을 설명하는 문장만으로, 추가 학습 없이 Stable Diffusion에서 결함이 있는 사진을 만든다. 정상 사진에 노이즈를 조금만 섞어서 출발하기 때문에(200스텝 중 마지막 60스텝만 진행) 물체의 모양과 배치는 거의 그대로 남고, 결함 단어(`faded`, `crack`)가 그림에 강하게 반영되도록 주의(attention) 맵을 키우면서 결함을 그려 넣는다.

단어별 주의 맵을 그려 보니 물체 단어(`table`)는 물체 전체를, 결함 단어(`faded`)는 상판 가운데 좁은 곳을 비췄고, 실제로 그 자리에 얼룩이 생겼다. 결함 단어의 주의 맵이 결함 위치를 알려 주므로, 결함 이미지와 함께 "어디가 불량인지"를 나타내는 마스크도 얻을 수 있겠다고 생각했다. 이상 탐지 모델을 평가하려면 이런 픽셀 단위 정답이 필요하다.

하지만 만든다고 다 쓸 수 있는 것은 아니었다. 헤이즐넛 균열은 4장 중 2장만 그럴듯했고, 나머지는 배경에 엉뚱한 자국이 생기거나 정상과 거의 같았다. 탁자는 "햇빛에 바램"보다 얼룩에 가까웠고, PCB는 균열 대신 커넥터가 망가지거나 글자와 배선이 뭉개졌다. 생성한 결함 데이터는 사람이 보고 골라야 하고, 물체에 맞는 결함 문장을 고르는 것이 중요하다는 것을 알게 됐다.

개인 실험 요약

원본 코드는 건드리지 않았고, 실험 셀에는 `[개인 실험]` 표시를 했습니다.

1. **PCB에 맞는 결함 문장**: 원본의 `crack` 문장으로는 PCB에 균열이 생기지 않고 커넥터가 망가지거나 글자·배선이 뭉개지기만 해서, PCB에 실제로 생기는 결함을 문장으로 주면 나아지는지 궁금했습니다. "부품 누락(missing component)"과 "탄 자국(burnt mark)" 두 문장을 시드 0, 1로 2장씩 만들었습니다. 탄 자국 시드 1은 IC 주변이 까맣게 탄 듯한 자국으로 덮였고, 결함 단어의 주의가 몰린 자리와 같았습니다. 부품 누락 시드 0은 부품이 있던 자리가 빈 패드로 바뀌어 부품이 빠진 것처럼 보였습니다. 나머지 2장은 결함이 뚜렷하지 않아서, 4장 중 2장이 결함처럼 보였습니다(`crack`은 원본 셀 4장 중 0장). 원본 셀은 시드가 출력되지 않아 같은 시드로 비교하지 못했고 장수도 적어서 단정할 수는 없지만, 물체에 맞는 결함 문장을 고르는 것이 중요하다고 느꼈습니다. 또 같은 시드끼리는 문장이 달라도 기판 모양이 거의 같아서, 전체 모습은 시드가 정하고 결함이 생기는 자리는 결함 단어의 주의가 정한다는 것도 알게 됐습니다.

2. **생성 이미지를 AI로 걸러 내기**: 만든 결함 이미지를 사람이 일일이 고르는 대신 AI가 먼저 걸러 줄 수 있는지 궁금해서, 6주차에 만든 PaDiM(VisA pcb4 정상 사진으로 학습)으로 생성 이미지 8장을 채점했습니다. PaDiM은 VisA 테스트에서 AUROC 0.9824로 실제 불량을 잘 가려냈고, 비교용으로 넣은 정상 사진 3장(원본, 512×512로 바꾼 것, 결함 없이 SD의 VAE로 압축했다 복원한 것)은 모두 정상으로 판정됐습니다(14.69 / 14.58 / 19.18점, 기준 23.84). 그런데 생성 이미지는 8장 모두 불량으로 판정됐습니다(30.38~42.17점). 눈으로는 결함이 안 보이던 3장도 포함입니다. 히트맵을 보니 탄 자국과 바닥 긁힘은 정확히 가리켰지만, 8장 모두 생성할 때 다시 그려진 USB 커넥터에도 반응했습니다. PaDiM은 "진짜 정상과 다른 곳"을 찾을 뿐, 그게 의도한 결함인지 생성 과정의 흔적인지는 구분하지 못한다는 것을 알게 됐습니다. 그래서 자동으로 고르는 데는 쓸 수 없었지만, 사람이 어디를 봐야 할지 알려 주는 도구로는 쓸 만하다고 느꼈습니다.

어려웠던 것

주의(attention) 맵을 이해하는 것이 가장 어려웠다. 문장 속 단어 하나가 그림의 어느 자리에 영향을 주는지 나타낸 지도라는 것까지는 알겠는데, 이걸 키우면 왜 그 자리에 결함이 생기는지는 바로 와닿지 않았다. 단어별 주의 맵을 나란히 그려 보고, `faded`의 주의가 몰린 상판 가운데에 실제로 얼룩이 생긴 것을 보고 나서야 이해가 됐다.

생성된 그림이 결함인지 아닌지 판단하는 것도 어려웠다. 탁자의 얼룩은 "햇빛에 바램"이라고 보기 애매했고, PCB의 글자와 배선이 뭉개진 것은 결함인지 생성 과정의 흔적인지 헷갈렸다. 결함 이미지 1장에 3~4분이 걸려서 여러 장을 만들려면 기다리는 시간이 길었다. 하지만 교수님이 현장에서는 오래 걸리는 작업이 하루 단위로 걸린다고 말씀하셔서, 부담 갖지 않고 그냥 해 봤다.

좋았던 점

정상 사진 1장과 문장만으로, 추가 학습 없이 결함 이미지가 나오는 것이 신기했다. 주의 맵으로 결함 단어가 그림의 어디에 영향을 주는지 눈으로 확인할 수 있어서 좋았고, 개인 실험에서 PCB에 맞는 문장(탄자국)으로 바꾸니 실제 결함처럼 나와서 뿌듯했다. 코랩 Pro의 A100으로 돌려서 1장이 3분 남짓(187~197초)이면 끝난 것도 좋았다.

현장에 쓴다면

드문 불량 사진을 이 방법으로 만들어 불량 데이터를 보충하는 데 쓸 것 같다. 다만 결과가 들쭉날쭉해서 그대로 쓰지 않고 사람이 보고 쓸 만한 것만 고를 것이다. 고르는 일은 생성 결과를 분석하는 AI를 하나 더 두어 도움을 받으면 좋겠다고 생각했는데, 개인 실험에서 PaDiM으로 해 보니 결함이 생긴 자리는 잘 찾았지만 생성 과정에서 다시 그려진 부분까지 불량으로 봤다. 그래서 AI는 "어디가 달라졌는지"를 표시해 주는 1차 검사로 쓰고, 그게 실제 결함처럼 보이는지는 사람이 최종 판단하는 식으로 나눠 쓸 것이다. 결함 문장은 "균열" 같은 일반적인 말보다 그 제품에 실제로 생기는 결함으로 정하고, 시드를 고정해 기록해 두어 좋은 결과를 다시 만들 수 있게 할 것이다.

앞으로 해 볼 것

PaDiM이 생성 이미지를 모두 불량으로 본 이유가 커넥터처럼 매번 다시 그려지는 부분 때문이었으니, 마스크(`mask_image`)로 결함을 넣을 자리만 바뀌게 하고 나머지는 정상 사진 그대로 두면 PaDiM이 결함 자리에만 반응하는지 확인해 보고 싶다. 그렇게 되면 생성 이미지를 AI로 걸러 내는 방법이 실제로 쓸 만해질 것 같다.